Whiteboard Session: Drowsiness Detection System Teardown

Hey! Grab a coffee. I want to walk you through the guts of what we just built. It’s easy to look at a system that beeps when someone falls asleep and think, "Oh, it just looks for closed eyes." But the architecture under the hood is actually pretty slick. Let's break down the "Code Picture."

1. The Architecture & Models

First off, what are we using to actually "see" the face?

- **The Model: Google MediaPipe Face Mesh.** Under the hood, this is a lightweight BlazeFace neural network coupled with a 3D landmark regression model.
- **Why this one?** Originally, we played with OpenCV Haar Cascades, but they are awful at handling uneven lighting. Standard academic papers always point to `dlib` and its 68-point predictor. The problem? Dlib requires a massive C++ build chain and crawls at 2-5 FPS on edge devices like the Raspberry Pi. MediaPipe is purely Python-accessible, gives us a whopping 468 3D landmarks, and runs at a buttery 15-30 FPS on our target hardware. It's an absolute powerhouse for edge-computing.

2. The Math Under the Hood

Here’s where it gets fun. How do we translate a web of 468 points into a "drowsiness" metric?

The Eye Aspect Ratio (EAR)

We use the Soukupová and Čech formula. Imagine 6 points drawing a polygon around the eye. We calculate the Euclidean distance between the top/bottom eyelids (the vertical) and the corners of the eye (the horizontal).

$$ EAR = \frac{||P_1 - P_5|| + ||P_2 - P_4||}{2 \cdot ||P_0 - P_3||} $$

- **The Trick:** By dividing the vertical distances by the horizontal distance, the ratio becomes *scale-invariant*. It doesn't matter if you are sitting 2 feet from the camera or 5 feet away; the math holds up perfectly.

The Exponential Moving Average (EMA)

Cameras are noisy. If we just plotted raw EAR, the graph would look like a seismograph during an earthquake. To fix this, we apply an EMA to smooth out the signal:

$$ S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1} $$

Where Y_t is the current EAR, and α is our smoothing factor. This stops the alarm from triggering just because the camera dropped a single frame.

The Degradation Index (The Clever Part)

This is my favorite part. Counting closed frames is a reactive strategy—by the time the alarm goes off, the car is in a ditch. We built a proactive strategy.

We measure the *speed* of your blinks in frames. We take a baseline (B) during the first 5 blinks. Then, we track your current blink speed (C).

$$ Degradation = \frac{C - B}{B} $$

If your normal blink is 5 frames, but your current blink is 15 frames, your degradation is $200%$. You are getting sluggish, and we can catch you *before* you actually fall asleep.

3. Function-by-Function Teardown

Let's look at the actual code structure. What's the mission of each function?

- **`FaceEyeDetector.detect()`: The Scout.** Its only job is to find the face, extract the exact x,y coordinates of the 6 points around the left and right eyes, and pass them down the chain.
- **`compute_ear()`: The Calculator.** It takes those 6 points and runs the EAR math we talked about above.
- **`BlinkDetector.update()`: The State Machine.** This is where the magic happens. It watches the EAR. When EAR drops below the threshold, it starts a stopwatch. When it rises again, it stops the stopwatch, logs a "blink", and calculates the Degradation Index.
- **`DrowsinessScorer.compute()`: The Judge.** It takes the raw EAR, the Degradation Index, and how long the eyes have been closed, and mashes them into a final "Drowsiness Score" from 0.0 to 1.0.

4. The Parameter Logic

If you open `drowsiness_detector_config.json`, you'll see a bunch of knobs and dials. Here's what they actually do:

- **`ear_threshold` (0.22)**: The tipping point. If EAR goes below 0.22, we consider the eye "closed." Change this to 0.30, and it'll think you are blinking even when you are just squinting.
- **`degradation_weight` (0.60)**: This controls our scoring formula. 60% of the drowsiness score comes from *how sluggish* you are, and 40% comes from *how closed* your eyes currently are. If you set this to 0.0, you disable the predictive magic entirely.
- **`min_blink_speed` (3)**: This is our noise filter. If a "blink" lasts less than 3 frames, we throw it out. Humans literally can't blink that fast, so we know it was just a camera glitch.
- **`calibration_blinks` (5)**: How many blinks we need to figure out your natural speed. Setting this too high means the system takes forever to start protecting you.

5. The "Small Things"

It's the edge cases that separate a toy from a production system.

- **Sustained Closure Boost:** Here’s a tricky edge case: What if the driver just closes their eyes and keeps them closed? Since they aren't completing a "blink," the degradation index sits at 0%. To fix this, if the eyes are closed for a sustained period, we *override* the math and forcibly push the drowsiness score to 100%.

- **Graceful Hardware Degradation:** We are triggering a physical 5V buzzer on the Raspberry Pi via the `RPi.GPIO` library. But you can't install that on a Windows laptop. We put a `try/except` block around the hardware imports so that if you run it locally, it silently falls back to printing `[BUZZER ON]` to the console instead of crashing.
- **State Carry-Forward:** If the neural net loses your face for exactly one frame because of a weird shadow, we don't assume your eyes closed. We carry forward the last known EAR value. It prevents false micro-blinks.

That’s the system in a nutshell. It’s lean, mathematically sound, and designed specifically to run reliably without an internet connection.